While this hook manages a single input and interacts with a specific backend endpoint, its design allows for reusability in different parts of our application. That's the power of the custom React hooks in action.

2.5.5 The backend: Handling API communication

The backend is the critical middle layer that connects our frontend to the AI model. Its primary responsibility is to serve as a secure, reliable intermediary between the user interface and the external AI service. This architecture allows us to handle sensitive information such as API keys on the server side, away from the user's browser. Let's explore the key functions of this backend service.

By default, the project uses the Google variant of server.js because of its robust conversation support and developer-friendly API and because it offers a free usage tier that covers the examples in this book. However, users can easily switch to the OpenAI variant, which also offers excellent multimodal capabilities. Please be aware that using OpenAI requires you to provide a credit card and will incur usage-based costs, as it no longer offers a free tier for general API access. You can switch to the OpenAI variant by starting the project with the following npm command:

```
npm run start:openai
```

This command runs the backend using the OpenAI server.openai.js variant, enabling you to use OpenAI tiers and features. This component performs the following key functions:

- *Environment variables*—Before any processing occurs, the backend retrieves essential credentials, like the OpenAI/Google API key, from environment variables. These variables are typically stored in an .env file to enhance security and avoid hardcoding sensitive information in the code itself. Users should create an .env file in the root directory of the project before running the application and add it to their .gitignore as well to prevent accidentally committing sensitive information to version control.
- *Validation*—The backend first receives a POST request from the frontend, likely triggered by user interaction with the chat interface. It parses the request body as JSON to extract the user's input text and performs any necessary validation checks to ensure valid data is processed.
- *Interaction with OpenAI/Google API*—Once the user input is validated, the backend uses the OpenAI/Google client library to interact with the OpenAI/Google API. It sends a request containing the user's input and additional configuration details.
- *Postprocessing*—Upon receiving a response from the OpenAI/Google API, the backend extracts the generated content and constructs a message object.
- *Response transmission*—The backend transmits the constructed message object back to the frontend application as a JSON response. This response will be